GitHub https://github.com/deveshsawant05/orderdesk

OrderDesk technical design

Company A Hyderabad ordering and kitchen operations
Implementation review dated 9 November 2026

Customer https://orderdesk-frontend.vercel.app/
Operations https://orderdesk-operations.vercel.app/
API health http://16.178.180.29:8000/health

Purpose and delivered slice

OrderDesk demonstrates a direct ordering flow from a customer's food selection through simulated payment into the correct outlet's live kitchen queue. The same backend owns menu prices, stock, permissions and order transitions. This document explains the implementation and its tradeoffs; it is not a claim of production readiness or a legal compliance certification.

Core and optional depth

The core is the ordering menu, location picker, cart and test payment, order tracking, central menu administration and outlet kitchen display. Optional depth includes outlet reports, central manager aggregation, saved mobile history, party requests and a rotating menu display. Delivery-app integration, inventory replacement and an owned rider application remain outside scope.

Reading the implementation

Code is split into frontend, operations and backend. deployment.md provides the short local setup and public fictional reviewer accounts. Live links and the EC2 address are deployment information supplied by the project owner; they are not an independent availability audit. The auto-assigned EC2 IP can change after stop/start.

System architecture

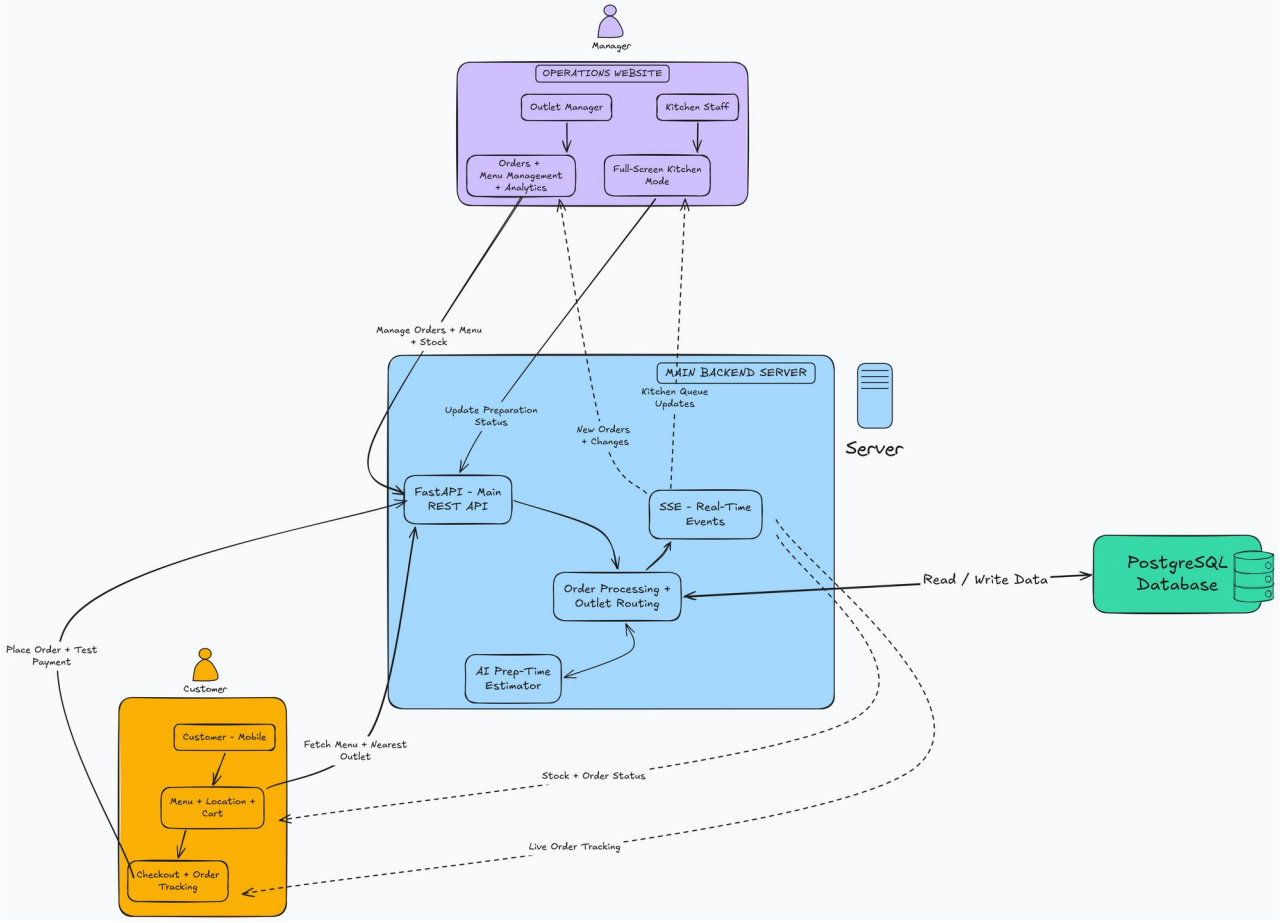


Figure 1 Request and event flow supplied in docs/architecture.png

How to read the diagram

Solid arrows represent commands and reads; dashed arrows represent server-sent events. FastAPI validates a command, updates PostgreSQL and records an event in the transaction. Browsers receive a lightweight event and refetch authoritative state. The prep estimator is an in-process component, not a separate AI service.

Deployment detail beyond the diagram

The two Next.js sites deploy separately on Vercel. Each rewrites its own /api path to the shared API. For the current simple demo, the API and PostgreSQL are containers on the same Sydney EC2 instance. PostgreSQL is a logically separate service even though it shares a host. Docker's network connects them; a named volume retains data. Public OSM tiles, Photon search and OSRM routing are external dependencies omitted from this simplified request-flow figure.

The diagram does not imply microservices, a message broker, a live rider feed or managed database redundancy. None of those are part of the current deployment.

Architectural choices and tradeoffs

Two Next.js applications

Customer ordering and staff operations have different navigation, permissions and screen priorities. Separate folders, package locks and Dockerfiles allow independent Vercel deployments and prevent the operations shell from appearing in the customer site. The cost is duplicated UI helpers, styles and assets; a shared package is a sensible next refactor. Next.js provides routing, React UI composition and initial server rendering of the customer menu; this is useful for a fast first menu view without moving transaction logic into either frontend.

Same origin API proxy

Browser fetch and EventSource use /api on the frontend origin. Next rewrites forward requests to API_INTERNAL_URL. This avoids putting a bare IP into browser URLs and keeps host-only cookies associated with the relevant site. The destination is built into rewrite configuration, so changing it requires a frontend rebuild. This simplifies browser integration but adds a proxy hop; streaming and reconnect behavior must be tested through Vercel. An HTTP backend destination is still unencrypted on that server-to-server leg.

One FastAPI backend

Pricing, outlet eligibility, payment simulation and state transitions live in one Python service so both apps follow the same rules. Pydantic schemas reject malformed inputs and SQLAlchemy controls transactions. Python also hosts the small numerical estimator without a separate model server. This modular monolith suits a bounded assignment and a small operating team; it avoids distributed transactions and independently deployed services that the core flow does not need.

PostgreSQL and Alembic

Relational constraints fit orders, line items, payments, users and outlet stock. Integer paise avoid floating-point currency errors. JSON snapshots preserve item choices and historical prices while allowing varied portions/add-ons. Transactions, row locks, uniqueness constraints and version checks protect checkout and concurrent staff actions. Alembic versions schema changes, including party ownership and fulfilment states. JSON fields are convenient but still require application validation.

Docker and the small AWS deployment

Compose makes the same services reproducible locally without host-side Node, Python or PostgreSQL installation. Hosting API and database together on EC2 avoids RDS network setup for this demo; the default VPC supplies AWS networking. A single host is a failure domain, and local volumes are not backups. A real rollout should move toward managed PostgreSQL, backups with restore drills, observability and an HTTPS API origin before personal customer data is used.

Data and transactional flows

Data model

Users have a role, optional outlet assignment and central-management permission. Sessions store hashes of opaque cookie tokens, expiry and optional verified phone identity. Outlets store coordinates, radius, minimum prep time and licence data. Categories, menu items, availability per outlet and coupons describe the catalogue. Carts hold selections and location. Payments record simulation attempts; orders and order items preserve the accepted commercial snapshot. Events, notifications, estimates, model coefficients and synthetic training samples support live operations and estimation. Party requests have their own contact, schedule, item quantities, owner and lifecycle. Loyalty accounts and transactions exist as backend depth.

Location and menu

The customer searches for a Hyderabad place or moves a pin on an actual OSM map. Eligibility uses Haversine radius distance to an open outlet, with a stable outlet-ID tie break. This is the current product decision: road distance is used after ordering, not to reject an address during selection. Menu availability combines publication, outlet stock, opening state and serving hours. Geocoder failure still permits manual map selection.

Cart and payment

Portions, add-ons, coupon, coordinates and menu versions are validated server-side. A payment attempt snapshots contact and commercial inputs; confirmation rechecks stock and pricing. Locking and an idempotency key prevent repeated confirmations from creating duplicate orders. A unique daily ticket number is allocated per outlet and IST date. UPI/card success, failure and cancellation are simulated; no gateway charges money. Prices include 5% GST, extracted as 5/105 from the applicable discounted amount using integer paise.

Immediate orders

Successful confirmation creates a new ticket directly in the kitchen. A manager or assigned kitchen staff can move new → preparing → ready → handed off. New/preparing orders can be cancelled subject to role/reason rules; reversal is simulated. A version conflict rejects stale actions. Immediate orders never require party-style manager approval. Live queues exclude terminal tickets and orders outside the current 12-hour operational window; managers retain a history view.

Party orders and history

Party requests require at least six hours' notice and enter pending. Only a manager accepts or declines. Accepted parties use a separate accepted → preparing → ready → fulfilled flow and do not become paid immediate-order tickets. Final quote/payment is outside this request workflow. They appear in the customer's separate party history and the kitchen's scheduled section; that section is absent when empty. Ownership supports the original session, and permitted phone history recovers older matching requests. Customer-history queries never accept an arbitrary phone as an orders query parameter.

Live updates and delivery estimates

Why server sent events

The hard requirement is pushing order changes to kitchens. Staff actions travel over REST; the server mainly needs to send notifications in the other direction. SSE fits that asymmetric flow without WebSocket infrastructure. A committed PostgreSQL event log provides ordered IDs and replay via Last-Event-ID. Event creation uses a transactional advisory lock so concurrent commits cannot leave a replay cursor skipping a late event. Streams are scoped to an outlet, central permissions or an opaque tracking token.

Reconnects and failure behavior

Streams send heartbeats, check session expiry and poll for committed records about once per second. The frontend refetches canonical state after an event and uses a 15-second recovery refresh when disconnected. SSE is an invalidation signal rather than the order record itself. Version checks still protect commands when two devices act together. This DB-polling design is transparent and durable for a demo, but many simultaneous streams will need shared fan-out such as Redis or another event transport. Multi-worker scheduling also needs coordination.

Pre fitted preparation estimates

The demo uses pre-fitted preparation coefficients and configured dish/outlet timings. During initial sample seeding, backend/app/prediction.py fits a small regression once on generated data and stores its coefficients in PostgreSQL. Orders then reuse those fixed coefficients with current kitchen-load and item inputs. It is not an LLM, an external AI service, online learning or a model trained on real kitchen operations. The local database was checked: the stored demo model exists and saved order estimates use it. Its presence on the deployed EC2 database has not been independently verified.

Inputs and safeguards

The preparation calculation considers pending/preparing orders, queued and current quantities, the largest configured dish prep time and recent completion time. The coefficients stay pre-fitted; the result can change with those inputs. Missing or invalid model output selects a deterministic workload-based fallback. The promised preparation duration is the higher of the computed estimate and the configured outlet minimum. These are demo timing assumptions, not measured production accuracy or a confidence interval. The recent-completion input currently includes placement-to-ready time, unlike the dashboard’s start-to-ready prep metric.

Delivery timing and correction

Road delivery time is added separately after preparation. OSRM supplies the driving route and road distance; the code uses a fixed allowance of 5 + ceil(road distance in km × 3) minutes, equivalent to an assumed effective speed of 20 km/h plus a five-minute handoff buffer. For example, 4 km adds 17 minutes to the current ready promise. Once marked ready, the actual ready timestamp is used; after handoff, the handoff timestamp is used. This is a distance heuristic, not traffic prediction or rider GPS. A routing failure leaves arrival unknown. Staff ±5-minute prep adjustments update the promise and customer estimate through SSE. Orders over ten minutes late trigger a recorded manager alert and customer update; SMS/WhatsApp sending is mocked.

This kind of ordering product is primarily a transactional, event-driven system. Its value does not depend on extensive AI. Reliable stock validation, payments, scoped access and live queues matter more than adding an unvalidated model. Personalised AI reorders are not implemented.

Interface security and validation

Customer experience

Large local food photos, restrained terracotta/cream colours, compact prices and food marks support a short menu scan. The location picker supports search, zoom, pan, pinch and pin selection. Checkout remembers contact and previous address rather than repeatedly asking for them. The tracker follows the supplied delivery-app layout with an OrderDesk theme: remaining minutes, a route map, status progress and expandable receipt. Empty, loading and error states are explicit. The sub-60-second checkout target still needs measured user testing.

Operations and kitchen

Managers get compact order grids and on-demand detail dialogs, separate from the reporting dashboard. Central managers default to all outlets and may filter; branch staff cannot switch to another outlet. Kitchen tickets use large quantities, elapsed/remaining time, late colours and direct prepare/ready/handoff buttons suitable for touch. Scheduled parties use a distinct horizontal section after immediate tickets and vanish when no active party exists. Completed work stays out of live queues. Readability at two metres and heat/noise conditions require real tablet trials.

Menu and stock control

The CMS edits central items, prices by city, portions, add-ons, allergens, serving hours and publication. Per-outlet stock uses direct controls. Cart errors preserve dish names when an item becomes unavailable. The display screen rotates full-image dishes with price, food details, FSSAI placeholder and a customer-site QR; it has no progress or exit controls. Navigation enters fullscreen, Escape returns to dashboard, and direct-open fullscreen needs a first tap because browsers require user activation.

Security implemented

Staff passwords are hashed with Argon2; session cookies are HTTP-only, with configurable Secure and SameSite=Lax. Customer and staff cookies are separate, with approximately 30-day and 12-hour lifetimes. Backend role/outlet checks protect reads and writes independently of hidden controls. Origin checks, an X-OrderDesk mutation header, CORS allowlists, schema validation and parameterised queries reduce common misuse. Contact changes reset marketing consent. Transactional notices and optional marketing consent are distinct.

Security boundaries still open

OTP is deliberately skipped for the demo. Anyone knowing a phone can recover that phone’s history while this flag is enabled. Opaque tracking links are bearer capabilities and must not be shared carelessly. The reported API uses public HTTP and the in-process login limiter can group visitors behind a proxy. Real deployment needs verified phone sign-in, TLS to the API, shared proxy-aware rate limiting, individual staff provisioning, retention/deletion processes and operational incident controls. Displaying consent is not proof of DPDP compliance.

Validation evidence

Local verification passed 29 backend tests covering pricing, idempotency, role boundaries, routing/fallback behavior, party lifecycle and phone-history isolation. Thirteen browser scenarios were verified, including SSE timing updates, all-outlet switching, party fulfilment, fullscreen/Escape and hiding the empty party section. Both frontend production builds and lint/type checks passed. These are local results, not a live deployment audit or production load certification.

Assumptions limits and next steps

Explicit assumptions

A1 — Company A has no supplied brand assets, exact outlet addresses or licences. The 17 named neighbourhood pins, 5 km-style service radii, opening/minimum-time settings and invalid licence placeholder are fictional demo configuration.
A2 — Food photos, portions, prices, add-on rules, allergens, spice values and coupon/reward economics are sample data. They need operator and food-safety validation.
A3 — The brief's three order times are anchored to 2 November 2026 IST as historical snapshots; this reference date is an implementation assumption, not a date supplied by the brief.
A4 — Six hours' catering notice, radius-based eligibility, OTP skipping and separate customer/operations deployments follow later product decisions. They are assumptions/extensions relative to the original brief.
A5 — Preparation coefficients are pre-fitted on synthetic data; configured prep times, delivery speed and handoff allowance are demo assumptions. The arrival estimate adds road-distance delivery time to the ready promise and does not imply real traffic or kitchen accuracy.
A6 — The shared public demo password and fictional email accounts are reviewer conveniences. They are not an acceptable real-business identity scheme.
A7 — One Sydney EC2 host, the default VPC and Vercel projects are a low-complexity demo deployment choice, not a production availability design.

Deliberately incomplete scope

Payments and reversals do not contact a gateway. Transactional messages are stored/mock rather than sent through WhatsApp/SMS. There is no rider app, rider location feed, delivery-app sync, POS replacement or inventory accounting. Loyalty accounting and mock verification exist in the backend, but the checkout rewards/verification promotion was removed from the current UI; end-to-end reward redemption is not claimed. The reorder recommender is not delivered. Public geocoding/routing/tile services have external availability and usage constraints.

Capacity and measurement

The brief's 1,200 orders/hour and Android-tablet constraint influence compact queues, batched reads and direct controls, but the system is not certified at that production peak. Existing developer notes record a small synthetic arrival-rate smoke check; it does not establish sustained load, concurrent SSE fan-out or realistic device performance. Measure sustained checkout throughput, duplicate callbacks, reconnect storms, database pool saturation and physical-tablet rendering before selecting production capacity.

Next work in priority order

First secure identities and transport, replace fictional business data, and add off-instance backups with restore checks. Then validate the core flow with outlet staff, measure checkout time and perform representative peak-load tests. Next connect payment and delivery providers, introduce scalable event fan-out and monitoring, and implement individual staff management. Finally train/evaluate prep estimates on consented operational observations, quantify error by outlet/load/time, and add reorder suggestions that never modify the cart automatically.

This document describes the source implementation reviewed on 9 November 2026. The editable Word file is the authoring copy; export it to PDF after changing any links, deployment details or reviewer notes. Keep deployment.md and the final PDF together for submission, and preserve the project commit history.